Sprawozdanie : Projekt 3

Maciej Flaga

1 Omówienie metody

Chcemy rozwiązać równanie Poissona w dwóch wymiarach

$$-\nabla^2 V(x,y) = \frac{\rho(x,y)}{\epsilon} \tag{1}$$

Dyskretyzując równanie oraz spłaszczając 2D siatkę zgodnie z konwencją

$$[i][j] = [p] = [i * (ny+1) + j]$$

Możemy zapisać nasz układ równań w postaci macierzowej

$$\mathbf{Mv} = \frac{\rho}{\epsilon},$$

gdzie macierz $\mathbf{M}$ jest macierzą pięciodiagonalną z wyrazami: $(2/\Delta x^2 + 2/\Delta y^2)$ na głownej diagonali; $-1/\Delta y^2$ na sąsiednich pod i naddiagonalach; $-1/\Delta x^2$ na diagonalach odległych o $\pm(ny+1)$ węzłów od głównej diagonali. Wzór na relaksację Gaussa-Seidla takiego równania macierzowego:

$$v_p^{k+1} = \frac{1}{D}\left(\sum_{q<p} M_{pq} v_q^{k+1} + \sum_{q>p} M_{pq} v_q^k + \frac{\rho_p}{\epsilon}\right),$$

gdzie D oznacza diagonalne elementy macierzy M. W ten sposób, w każdym kroku przestrzennym korzystamy z rozwiązania z tej samej iteracji. W rzeczywistości, kod nie działał na żadnych macierzach a jedynie emulował takie mnożenie działając na wzorze:

$$v_p^{k+1} = \frac{1}{D}\left(\frac{v_{p-ny-1}^k + v_{p+ny+1}^k}{(\Delta x)^2} + \frac{v_{p-1}^k + v_{p+1}^k}{(\Delta y)^2} + \frac{\rho_p}{\epsilon}\right)$$

Z powyższego wzoru, widzimy że aktualizując dany węzeł korzystamy z informacji zawartych w jego najbliższych czterech sąsiadach oraz z jego samego. Propagację informacji w algorytmie Gaussa-Seidla można zoptymalizować dzieląc siatkę we wzór szachownicy i aktualizując na zmianę białe i czarne pola. W ten sposób każde pole korzysta z najnowszych informacji. Ponadto żadne białe pole nie korzysta z informacji z żadnego innego białego pola (poza sobą) co czyni taki algorytm świetnym do zrównoleżnienia. Z tego powodu zdecydowano się napisać kod w CUDA C++ aby większość obliczeń przeprowadzić na karcie graficznej, której architektura została stworzona z myślą o obliczeniach równoległych. Kolejną optymalizacją jaką możemy zastosować jest wprowadzenie metody SOR, która działa na podobnej zasadzie jak algorytm GS. Jedyną zmianą jest wykorzystanie szybkości zmian w poszczególnych punktach między iteracjami i na ich podstawie napędzać dalsze zmiany. Wzór opisujący taką relaksację:

$$v_p^{k+1} = (1-\omega)v_p^k + \frac{\omega}{D}\left(\frac{v_{p-ny-1}^k + v_{p+ny+1}^k}{(\Delta x)^2} + \frac{v_{p-1}^k + v_{p+1}^k}{(\Delta y)^2} + \frac{\rho_p}{\epsilon}\right), \qquad \omega \in (1,2) \tag{2}$$

Zauważmy, że dla $\omega = 1$ dostajemy wzór algorytmu GS.

2 Realizacja

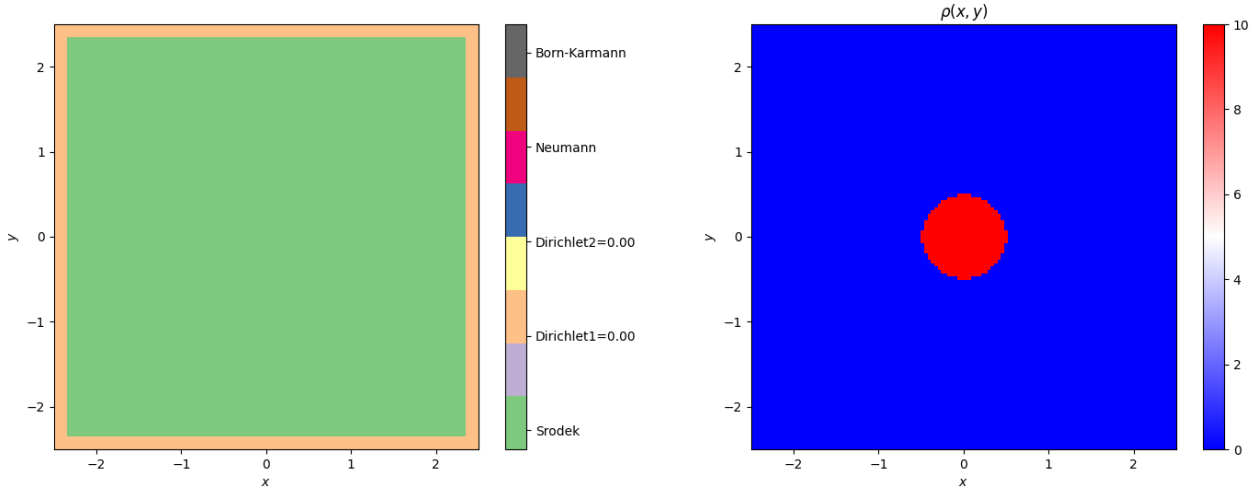
Napisano funkcję `SOrelaxHalf`[1], która implementuje wzór (2) dla węzłów o określonej parzystości, z której następnie korzystają funkcje `SO_relax_full_once` oraz `petla_relaksacyjna`. W złożeniu, trzy funkcje nadpisują spłaszczoną macierz `v` dla danych parametrów układu, ω i tolerancji błędu `tol`. Techniczny przebieg kodu:

```
(nwenv) maciej@konkuter:~/kod/sem6/uklnw/nwpyt/lab3$ make
nvcc main.cu -arch=native -Xcompiler -Wall,-Wextra -Werror all-warnings -O3 -lm -o main.out
./main.out
UKLAD PIERWSZY:
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.00; tol=1e-09; iteracje=itmax=100000; czas=10.522s
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=30000; czas=3.339s
(Lx,Ly)=(10.0,10.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=30000; czas=3.23s
(Lx,Ly)=(15.0,15.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=30000; czas=3.233s
(Lx,Ly)=(20.0,20.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=30000; czas=3.226s
UKLAD TRZECI:
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=8000; czas=0.577s
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=8000; czas=0.59s
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=8000; czas=0.588s
(Lx,Ly)=(5.0,5.0); (nx,ny)=(511,511); omega=1.90; tol=1e-09; iteracje=8000; czas=0.581s
UKLAD DRUGI:
(Lx,Ly)=(10.0,5.0); (nx,ny)=(1023,511); omega=1.90; tol=1e-09; iteracje=57000; czas=8.736s
(Lx,Ly)=(10.0,5.0); (nx,ny)=(1023,511); omega=1.90; tol=1e-09; iteracje=49000; czas=7.692s
(Lx,Ly)=(10.0,5.0); (nx,ny)=(1023,511); omega=1.90; tol=1e-09; iteracje=56000; czas=8.661s
(Lx,Ly)=(10.0,5.0); (nx,ny)=(1023,511); omega=1.90; tol=1e-09; iteracje=59000; czas=9.109s
python plot.py
```

Parametrem służącym jako warunek stopu została zmiana normy „wektora“ (spłaszczonej macierzy) `v` między kolejnymi iteracjami (musi być mniejsza niż `tol`). Możemy zauważyć, że liczba iteracji jest wielokrotnością 1000. Wynika to z faktu, że przesył danych między pamięcią karty graficznej i RAMu jest kosztowny lecz konieczny do sprawdzenia błędu. Z tego powodu błąd sprawdzany jest co daną liczbę iteracji sterowaną parametrem `co_ile` przyjętym jako 1000.

2.1 Układ pierwszy

Opisany następującymi mapami:

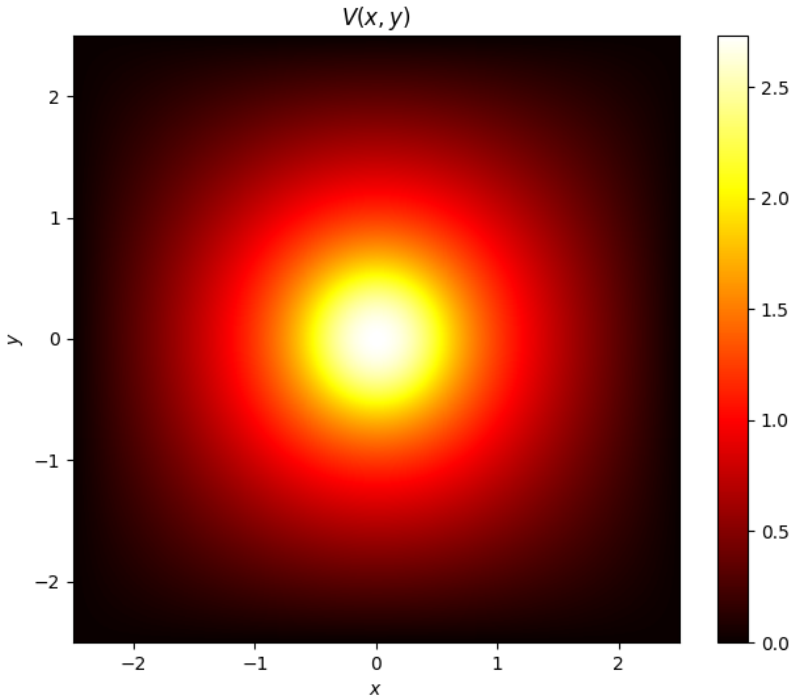


(a) Mapa warunków brzegowych

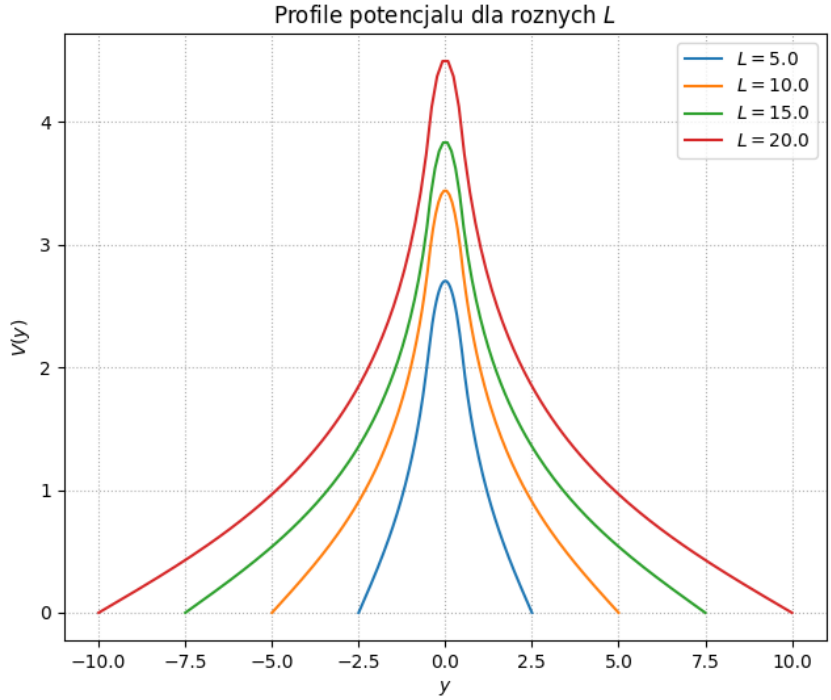
(b) Mapa gęstości ładunku

[1]Warto zaznaczyć, że skorzystano z pomysłu modelu językowego Google Gemini 3 Pro aby rozróżnić typy węzłów na poszczególne rodzaje warunków brzegowych. W ten sposób cały kod opiera się na uniwersalnej funkcji relaksującej.

Rozwiązanie:



Zbadano również zmianę profilu potencjału wzdłuż prostej $(x = 0, y)$ dla różnych wartości obszaru roboczego L.

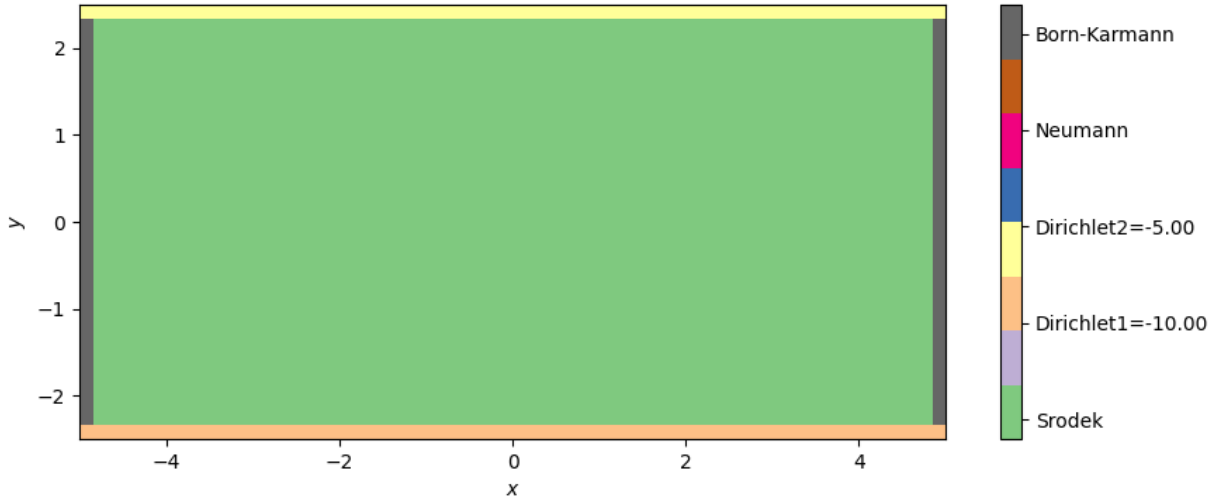


Numeryczne rozwiązanie wydaję się być zgodne z intuicją. Widzimy, że dla większych wartości L potencjał opada wolniej ponieważ wpływ brzegów Dirichleta jest mniejszy i większe znaczenie ma rozkład gęstości ładunku w środku układu. Z informacji w terminalu możemy również odczytać ogromny zysk w wydajności z zastosowania schematu SOR

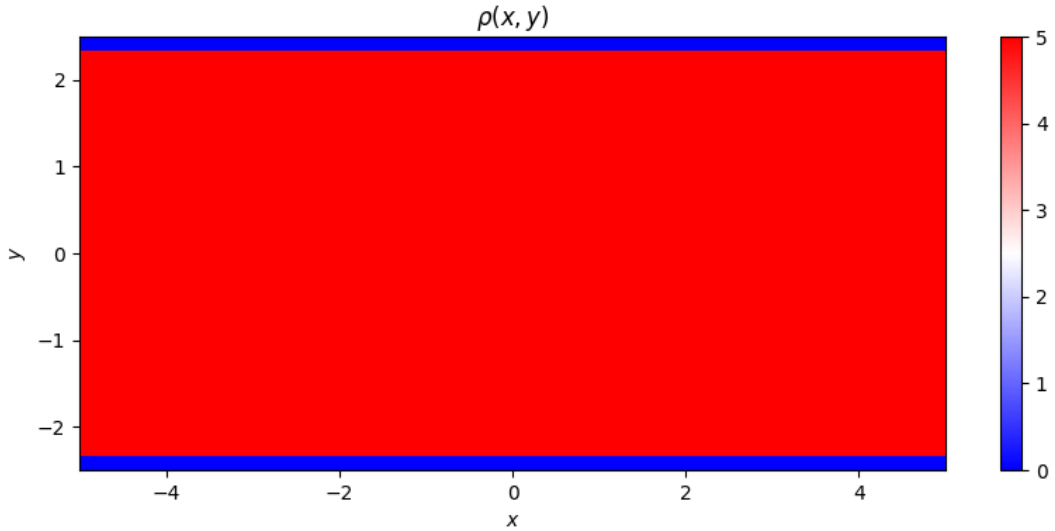
ponad GS. Dla klasycznego schematu GS program osiągnął górny limit iteracji. Stosując schemat SOR otrzymaliśmy co najmniej trzykrotne polepszenie wydajności przy $\omega = 1.9$. Z tego powodu dalsze obliczenia prowadzone są dla $\omega = 1.9$.

2.2 Układ drugi

Opisany mapami:

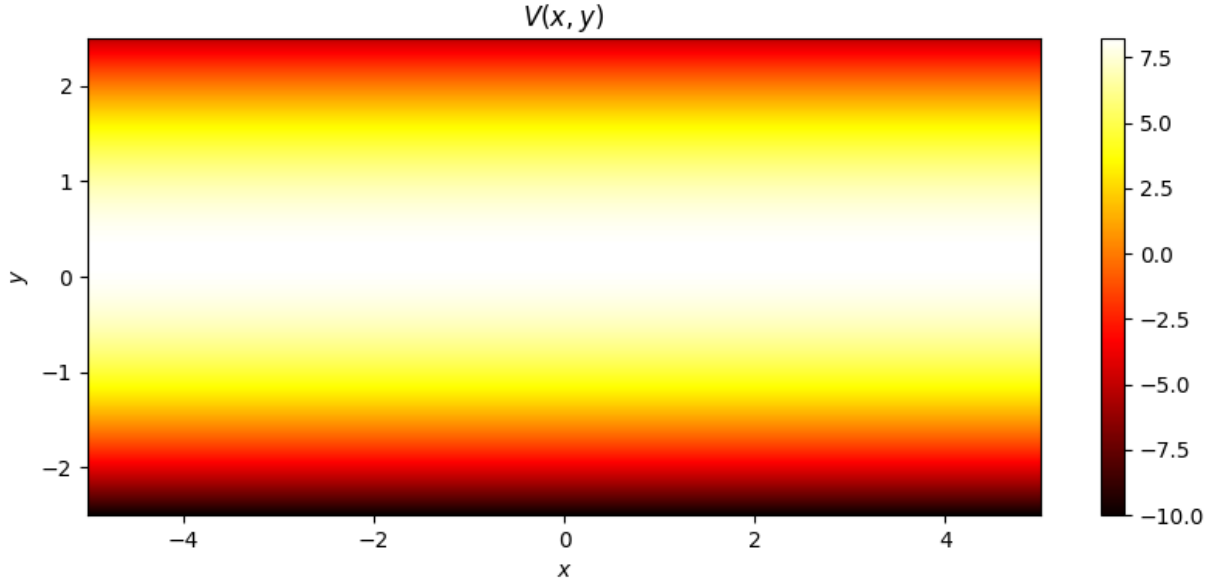


(a) Mapa warunków brzegowych

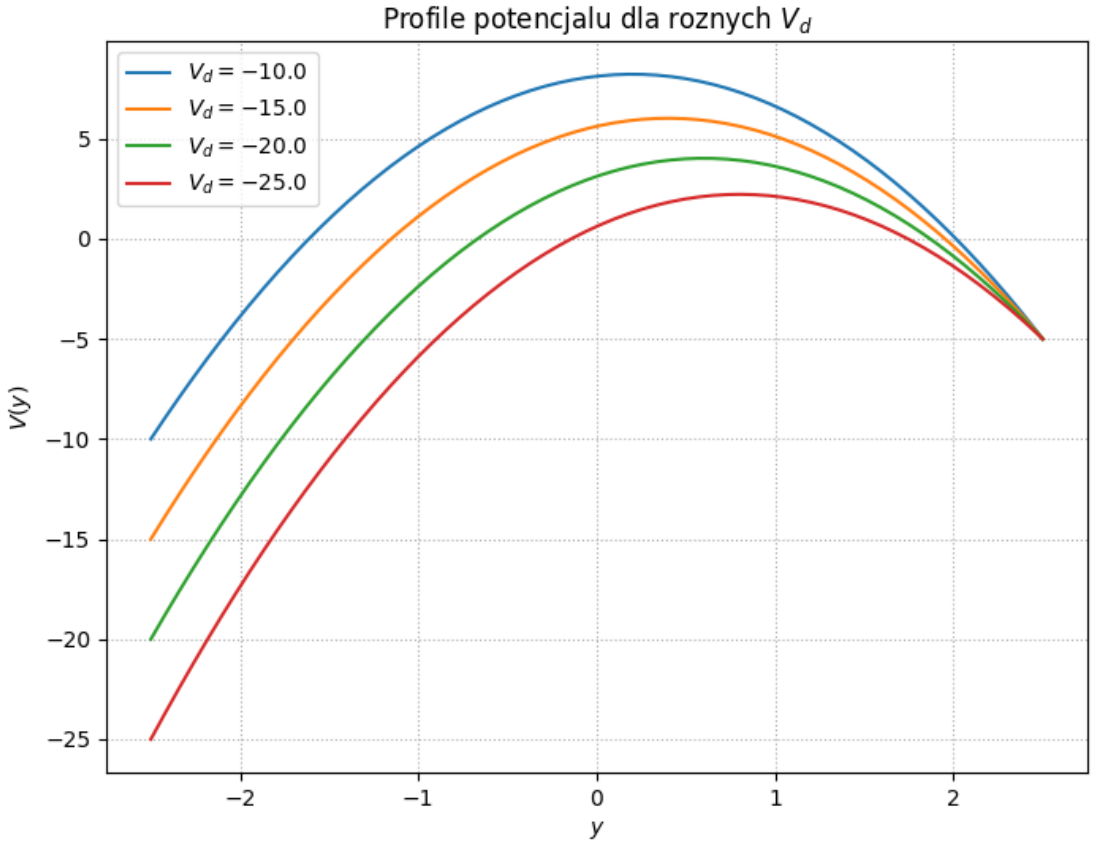


(b) Mapa gęstości ładunku

Rozwiązanie:



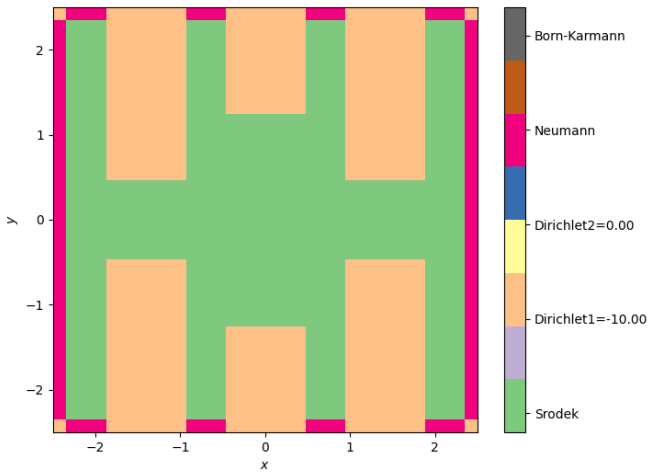
Zbadano profile potencjału wzdłuż prostej $(x = 0, y)$ dla różnych wartości potencjału na dolnej krawędzi:



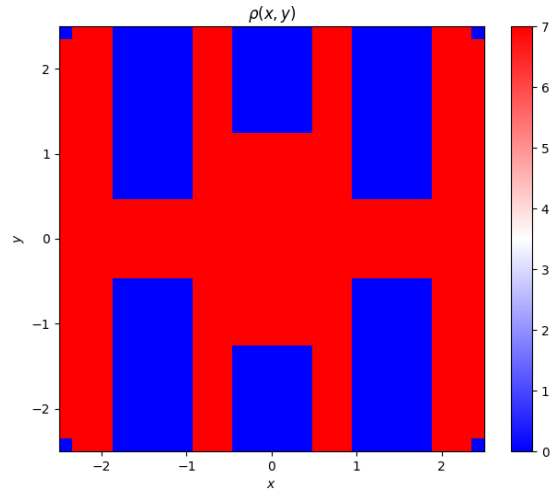
Podobnie widzimy, że rozwiązanie jest zgodne z intuicją. Wraz ze spadkiem potencjału V_d obserwujemy spadek średniego potencjału w całym przekroju drutu kwantowego oraz przesuwanie się maksimum w stronę górnej krawędzi.

2.3 Układ trzeci

Opisany mapami:

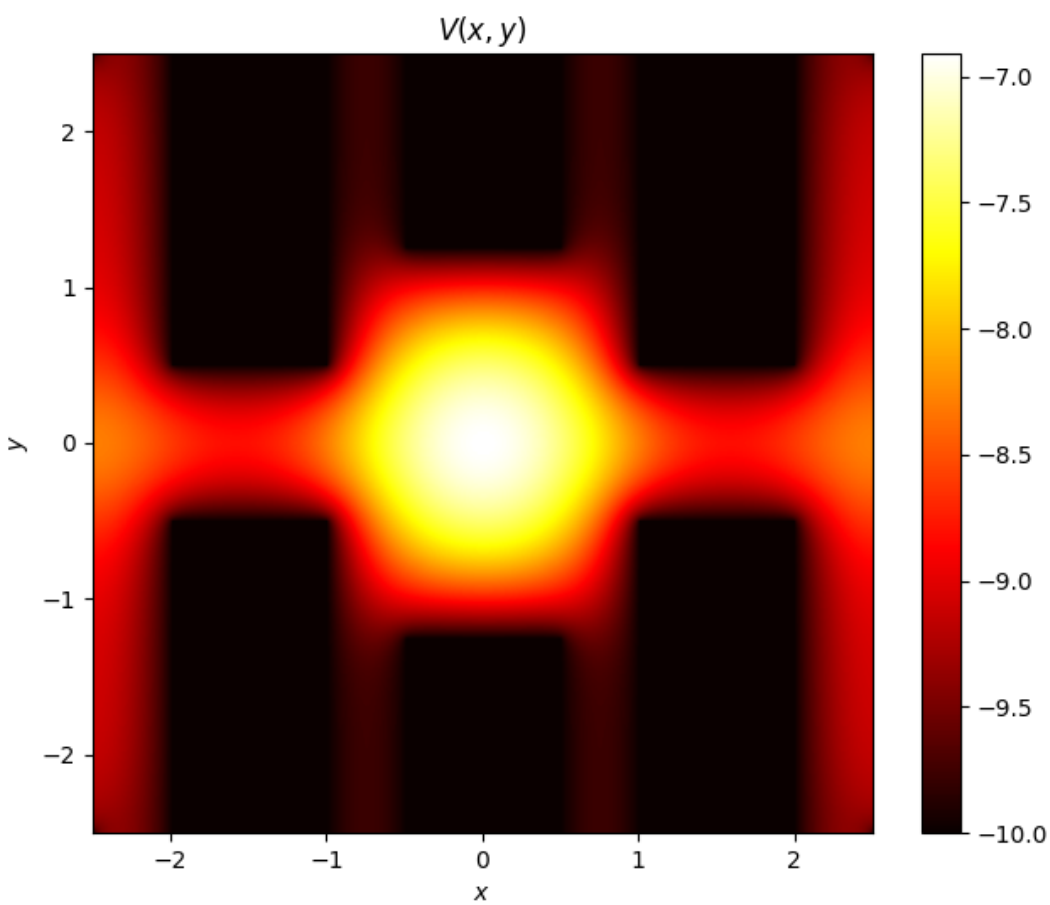


(a) Mapa warunków brzegowych

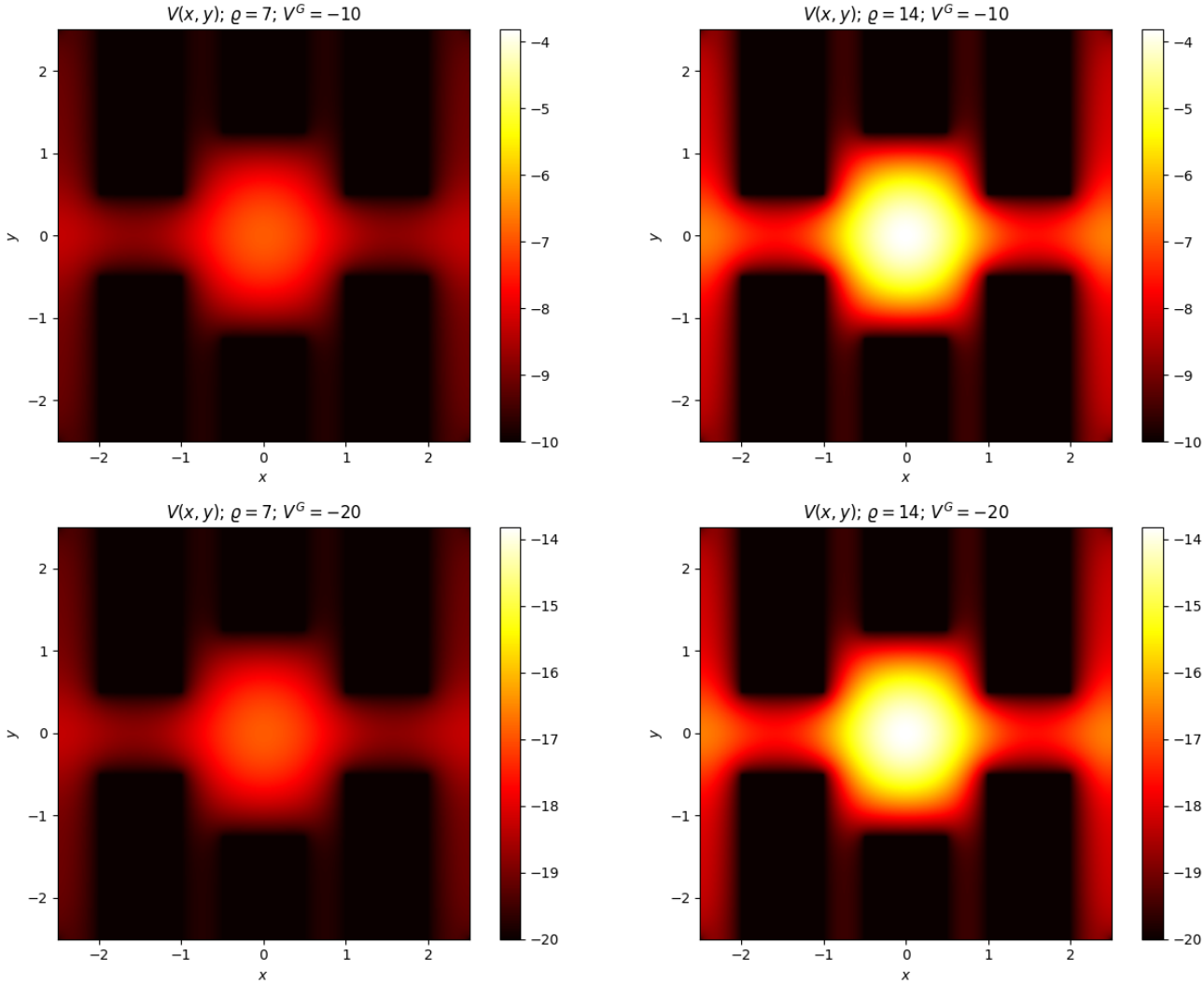


(b) Mapa gęstości ładunku

Rozwiązanie:



Zbadano wpływ wartości ρ oraz V^G na rozkład potencjału:



W środku układu powstaje maksimum lokalne potencjału. Widzimy, że dla wyższych wartości ρ potencjał przesuwa się w stronę wyższych wartości wszędzie co można uzasadnić przemnożeniem równania (1) przez jakąś stałą, coś na zasadzie liniowej odpowiedzi. Podobną sytuację obserwujemy zmieniając wartość V^G, widzimy że mapa potencjału wygląda tak samo tylko przesunięta o wartość -10. Uzasadnienia na to nie wymyśliłem. Wraz z zmianą tych dwóch parametrów wysokość maksimum się zmienia lecz sam rozkład potencjału nie zmienia swojego kształtu zauważalnie.

3 Źródła

Model Anthropic Claude Opus 4.7 / Sonnet 4.6 wygenerował skrypty do niektórych wykresów w Pythonie.